

1. Generalización simbólica (reglas del lenguaje)

Se pueden declarar tipos de forma explícita con : TypeName (ej: const user: User = {...})

Se usan interface para definir la "forma" que debe tener un objeto (qué propiedades tiene y de qué tipo)

Tiene tipos primitivos heredados de JS (string, number, boolean, null, undefined, etc.) y propios (any, unknown, never, void)

Permite combinar tipos con uniones (tipo1 | tipo2) para decir que algo puede ser de más de un tipo

Permite genéricos (Array<string>) para especificar qué contiene una estructura

Funciona con clases e interfaces juntas (POO), definiendo tipos en constructores, parámetros y retornos de funciones

Si el código no cumple estas reglas, el compilador tira un error antes de ejecutar (comprobación estática de tipos)

2. Creencias de los profesionales

Al ser fuertemente tipado, detecta errores antes de ejecutar el programa (cosa que en JS puro no se puede, porque su tipado es débil y dinámico)

Resalta comportamientos inesperados en el código, reduciendo la posibilidad de bugs

Como ya sabés JS, "ya sabés" casi todo TypeScript → se cree que es fácil de adoptar sin perder lo que ya sabías

Permite trabajar con conceptos de lenguajes más "serios" (tipado, interfaces, genéricos, clases abstractas) que hacen el código más prolijo y mantenible, sobre todo en proyectos grandes